Team Introduction: The Knowledge Graph Projects

Team Introduction: The Knowledge Graph Projects

Thomas Jones

This document walks contributors through the two repositories that make up the enterprise knowledge graph initiative: the platform (hc-enterprise-kg) that defines the engine and schema, and the content repository (hc-cdaio-kg) that contains the actual entities, relationships, and due-diligence analysis for a real public company. Understanding the boundary between them is essential for successful contribution to the project.

Publication Date: 2026

https://www.thehipsterciso.com

Contents

1 The Big Picture 1

2 Repository 1: hc-enterprise-kg — The Platform 2

2.1 What It Does 2

2.2 Architecture at a Glance 2

2.3 Key Source Directories 3

3 Repository 2: hc-cdaio-kg — The Content 4

3.1 Scale 4

3.2 How the Content Is Organized 4

3.3 Evidence Classification 4

3.4 Provenance Requirements 5

4 How the Two Projects Relate 6

5 Installation 7

5.1 macOS and Linux 7

5.2 Windows 7

5.3 What the Installer Does 7

6 Your Day-to-Day Workflow 9

6.1 The Fast Path: Claude Cowork 9

6.2 The Manual Path 9

6.3 What Blocks a Commit vs. What Warns 10

6.4 Automation Pipeline 10

7 Working with the Knowledge Graph Through AI 11
7.1 What Cowork Can Do That Claude Desktop Alone Cannot 11
7.2 Available MCP Tools 12
8 Why This Approach Matters for the CDAIO Program 13
8.1 Instant Situational Reporting 13
8.2 Gap Analysis and Enrichment Recommendations 13
8.3 Adding and Enriching Without Manual Effort 14
9 Questions for the CMU CDAIO Practicum 16
9.1 Deliverable 1: Company Overview and Current State 16
9.2 Deliverable 2: Capability and Maturity Assessment 16
9.3 Deliverable 3: Data and AI Strategy 17
9.4 Deliverable 4: Strategic Plan for Enterprise Data Management 17
9.5 Deliverable 5: Final Presentation 18
10 Key Principles to Internalize 19
11 Questions? 20

1 The Big Picture

We are building a queryable, auditable knowledge graph of an enterprise — not another PDF that sits in SharePoint until the next deal cycle. The work originated as a Carnegie Mellon CDAIO program practicum and has grown into a demonstration of what rigorous open-source intelligence, AI-assisted analysis, and structured data modeling can produce without access to proprietary documents.

Two repositories power this initiative. One is the **platform** (the engine, the schema, the tooling). The other is the **content** (the actual entities, relationships, and due-diligence analysis for a real public company). Understanding the boundary between them is the single most important thing you can internalize on day one.

2 Repository 1: hc-enterprise-kg — The Platform

Repository: github.com/thehipsterciso/hc-enterprise-kg (https://github.com/thehipsterciso/hc-enterprise-kg)

This is the open-source Python framework that defines what a knowledge graph *is* in our world. Think of it as the engine, the schema, and the quality controls — everything except the actual company data.

2.1 What It Does

The platform provides three core capabilities:

Synthetic generation. It can produce a realistic "digital twin" of an enterprise from scratch — people, departments, systems, vendors, data assets, compliance frameworks — scaled by industry profile and organization size. This is useful for testing, demonstrations, and scenario planning before you commit real data.

Schema and validation. It defines 30 entity types and 58 relationship types, all enforced through Pydantic v2 domain models with contextual metadata like dependency strength, risk effectiveness, and confidence scores. Every entity extends `BaseEntity` (UUID, type, name, description, tags, timestamps, versioning). Every relationship carries weight and confidence, not just a label.

Query and analysis. The engine supports traversal (neighbors, shortest path, blast radius), centrality computation, fuzzy search, and statistics — all exposed through a CLI, a REST API, and a Model Context Protocol (MCP) server that integrates directly with Claude Desktop.

2.2 Architecture at a Glance

The system flows through a clean pipeline:

```
CLI → Orchestrator → Generators → KnowledgeGraph → Export / Query / MCP Server
```

The knowledge graph itself sits on a NetworkX MultiDiGraph backend (Neo4j-swappable). Generation follows 11 sequential layers (L00 Geography through L10 Vendors) where each layer builds on previous ones to maintain referential integrity. After generation, the `RelationshipWeaver` enriches connections with contextual metadata, and `QualityAssessment` runs automated checks covering risk consistency, tech stack coherence, field correlation, and encryption alignment.

2.3 Key Source Directories

TABLE 2.1: PLATFORM REPOSITORY DIRECTORY STRUCTURE

Directory	Purpose
`src/domain/`	Entity and relationship models (Pydantic v2)
`src/engine/`	Graph engine abstraction and NetworkX implementation
`src/synthetic/`	Orchestrator, generators, relationship weaver
`src/cli/`	Click-based command interface
`src/mcp_server/`	MCP integration for Claude Desktop
`src/analysis/`	Centrality, blast radius, traversal
`src/ingest/`	CSV/JSON ingestion with schema mapping
`src/export/`	JSON export (optimized for LLM context windows)

3 Repository 2: hc-cdaio-kg — The Content

Repository: github.com/thehipsterciso/hc-cdaio-kg (https://github.com/thehipsterciso/hc-cdaio-kg)

This is where the actual enterprise intelligence lives. Built on top of the `hc-enterprise-kg` platform, this repository contains the knowledge graph for a specific public company — constructed entirely from open-source intelligence (OSINT).

3.1 Scale

The graph currently contains:

- **2,900+ entities** across 26 types, organized in 11 architectural layers
- **6,000+ relationships** spanning 39 distinct types
- **401 structured analysis documents** covering 10 due-diligence stages
- Research ledgers tracking methodology and evidence confidence tiers

3.2 How the Content Is Organized

```
hc-cdaio-kg/
├── entities/           # One JSON file per entity type (26 files)
├── relationships/      # One JSON file per relationship type (39 files)
├── due-diligence/      # 10-stage structured analysis (401 JSON docs)
├── docs/               # Research ledgers, methodology documentation
├── scripts/            # Install, sync, build, quality automation
└── .github/workflows/  # CI/CD
```

The per-type JSON file structure is a deliberate design decision. Having one file per entity type and one per relationship type prevents merge conflicts when multiple team members are contributing in parallel. You will rarely touch the same file as a colleague unless you are working on the same entity type.

3.3 Evidence Classification

Every claim in this graph carries an explicit confidence classification:

- **FACT** — Directly sourced from public filings, official documentation, or verified records
- **INFERENCE** — Reasonably derived from multiple corroborating sources
- **HYPOTHESIS** — Analytical judgment requiring further validation

This is not optional metadata. It is how we distinguish what we know from what we think we know.

3.4 Provenance Requirements

Every entity you contribute must include provenance fields documenting where the data came from, who assessed it, when, and at what confidence level:

```
{
  "provenance": {
    "primary_data_source": "SEC 10-K FY2024, docs.example.com",
    "assessed_by": "Your Name",
    "last_assessed_date": "2026-03-02",
    "confidence_level": "0.90"
  }
}
```

This is non-negotiable. A knowledge graph without provenance is just a database with opinions.

4 How the Two Projects Relate

```
┌──────────────────────────────────────────────────────┐
│                  hc-enterprise-kg                    │
│                   (The Platform)                     │
│                                                      │
│  Schema ─── Validation ─── Engine ─── MCP Server     │
│    │              │           │            │         │
│    ▼              ▼           ▼            ▼         │
│  Defines       Enforces    Queries     Exposes       │
│  entity &      data        graph       tools to      │
│  relationship  quality     traversal   Claude        │
│  types                     & analysis  Desktop       │
└──────────────────────┬───────────────────────────────┘
                       │
                       │  provides schema, validation,
                       │  and query capabilities to
                       │
                       ▼
┌──────────────────────────────────────────────────────┐
│                   hc-cdaio-kg                        │
│                  (The Content)                       │
│                                                      │
│  Entities ─── Relationships ─── Due Diligence        │
│    │               │                  │              │
│    ▼               ▼                  ▼              │
│  2,900+         6,000+           401 structured      │
│  across 26      across 39        analysis docs       │
│  types          types            (10 stages)         │
└──────────────────────────────────────────────────────┘
```

The platform defines the rules. The content follows them. When you interact with the knowledge graph through Claude Desktop (via MCP), the platform's engine loads the content repository's JSON files and exposes them through queryable tools — search, traversal, blast radius, centrality analysis.

5 Installation

The install scripts live in the content repository (`hc-cdaio-kg`). They handle everything end-to-end: Python 3.11+ detection or installation, Git, Poetry, dependency resolution, knowledge graph build, MCP server registration with Claude Desktop, sync daemon setup, and personal branch creation.

You do not need to install anything manually.

5.1 macOS and Linux

```
git clone https://github.com/thehipsterciso/hc-cdaio-kg
bash hc-cdaio-kg/scripts/install.sh
```

5.2 Windows

```
git clone https://github.com/thehipsterciso/hc-cdaio-kg
powershell -ExecutionPolicy Bypass -File hc-cdaio-kg\scripts\install.ps1
```

5.3 What the Installer Does

The script runs through eight stages automatically:

1. **OS detection** — Identifies macOS (Intel/Apple Silicon), Linux (apt/dnf), or Windows
2. **Python ≥ 3.11** — Locates or installs a compatible Python interpreter
3. **Git** — Verifies availability, installs if missing
4. **Engine repository** — Clones `hc-enterprise-kg` (the platform) into your home directory
5. **Poetry** — Installs the dependency manager and binds it to your Python interpreter
6. **Python dependencies** — Runs `poetry install --extras mcp` to resolve the full dependency tree
7. **Knowledge graph build** — Constructs `graph.json` from the entity and relationship files in `hc-cdaio-kg`
8. **Claude Desktop MCP registration** — Registers the knowledge graph as an MCP server so Claude Desktop can query it immediately

After the core install, the script optionally sets up:

- **Sync branch** — Creates a personal `username/data` branch for your contributions
- **Automation daemons** — Installs scheduled tasks (LaunchAgents on macOS, systemd timers on Linux, Task Scheduler on Windows) for the 30-minute sync cycle, morning rebuild, and evening PR closure
- **Claude Desktop restart** — Gracefully restarts Claude to load the new MCP configuration

The script is idempotent. Run it again any time you need to update or repair your environment.

6 Your Day-to-Day Workflow

6.1 The Fast Path: Claude Cowork

For most contributors, the day-to-day workflow is a conversation. Open Cowork, point it at your local `hc-cdaio-kg` repository, and tell it what you want to do. Cowork handles the search-before-adding check, the entity or relationship creation via MCP, the validation, the commit, and the push — all from a single conversational thread. You do not need to touch the command line unless you want to.

Example: "I found that the company uses ServiceNow for IT service management. Add it as a system entity with status active and source the 2024 10-K, then create a 'depends_on' relationship from the help desk organizational unit." Cowork will search for duplicates, create the entity, validate the relationship, and commit the change to your personal branch.

6.2 The Manual Path

If you prefer working directly with the files, or if you are contributing outside of Cowork, the manual workflow is:

Step 1 — Search before adding. Run the validation script to check for existing entities before creating new ones. The system catches duplicates at an 85%+ similarity threshold.

```
python scripts/sre/validate-commit.py
```

Step 2 — Add your entity or relationship.#to the appropriate per-type JSON file in `entities/` or `relationships/`. Include all required fields: `id`, `name`, `entity_type`, `description`, and full provenance.

Step 3 — Validate relationships. Every relationship must reference existing entity IDs and use a valid `relationship_type` from the schema. Dangling references will block your commit.

Step 4 — Commit and push.

```
git add entities/ relationships/
git commit -m "feat: add new system entity for [description]"
git push origin your-feature-branch
```

Pre-commit hooks run automatically if you have installed them:

```
cp scripts/sre/pre-commit-hook .git/hooks/pre-commit
```

```
chmod +x .git/hooks/pre-commit
```

6.3 What Blocks a Commit vs. What Warns

TABLE 6.1: COMMIT VALIDATION RULES

Check	Blocks Commit?	Threshold
Schema compliance	Yes	100% required
Required fields	Yes	100% required
Relationship integrity	Yes	100% required
Provenance completeness	No (warning)	Strongly encouraged
Potential duplicates	No (warning)	85%+ similarity

6.4 Automation Pipeline

The repository runs an automation cycle that keeps everything synchronized:

- **Morning sync** rebuilds the unified graph from per-type files
- **30-minute intervals** split changes back into per-type files
- **Evening closure** opens pull requests to main

This means you work with the per-type files during the day, and the pipeline handles consolidation.

7 Working with the Knowledge Graph Through AI

The easiest and recommended way to work with the knowledge graph is through **Claude Cowork**. Cowork is a desktop application that combines the MCP knowledge graph tools with full file system access, code execution, git operations, and document generation in a single conversational interface. It is the primary workflow surface for this project.

What this means in practice: you do not need to learn command-line tools, memorize JSON schemas, or run validation scripts yourself. You describe what you want in natural language, and Cowork handles the mechanics — querying the graph, adding entities, creating relationships, running validation, committing changes, generating reports, and producing deliverables.

7.1 What Cowork Can Do That Claude Desktop Alone Cannot

Claude Desktop with MCP gives you query and mutation access to the knowledge graph. That is valuable, but it stops at the graph boundary. Cowork extends that reach across the entire workflow:

- **Query, add, update, and remove** entities and relationships through natural language — the same MCP tools available in Claude Desktop
- **Run validation and pre-commit checks** before committing, catching schema violations, dangling references, and duplicates without you ever running a script
- **Commit and push changes** directly to your personal branch, with properly formatted commit messages and staged files
- **Generate reports and documents** — Word documents, spreadsheets, presentations, PDFs — directly from graph data, without leaving the conversation
- **Search the web** for OSINT sources, then immediately enrich the graph with what it finds
- **Read and process uploaded files** — drop a 10-K filing, an earnings transcript, or a compliance audit into the conversation and let Cowork extract entities and relationships from it

The MCP server auto-reloads when the graph file changes, so any additions or modifications are immediately reflected in subsequent queries without restarting anything.

7.2 Available MCP Tools

Whether you use Cowork or Claude Desktop, the following tools are available through the MCP server:

- `search_entities` — Fuzzy name search across all entity types
- `get_entity` — Full detail for a single entity by ID
- `get_neighbors` — All entities directly connected to a given node
- `get_blast_radius` — Every entity reachable within N hops (impact analysis)
- `find_shortest_path` — Shortest connection between any two entities
- `compute_centrality` — Degree, betweenness, or PageRank scoring
- `get_statistics` — Entity and relationship counts, density, connectivity
- `list_entities` — Browse entities by type
- `add_entity` — Create a new entity with schema validation
- `update_entity` — Modify fields on an existing entity
- `remove_entity` — Delete an entity and all its relationships
- `add_relationship` — Create a validated relationship between entities
- `add_relationships_batch` — Atomic batch creation (all-or-nothing validation)
- `remove_relationship` — Delete a relationship by ID
- `find_most_connected` — Top entities by raw connection count

8 Why This Approach Matters for the CDAIO Program

This is not an academic exercise bolted onto a course requirement. The knowledge graph is a working demonstration of what a Chief Data and AI Officer actually needs to build: a structured, queryable, evidence-backed model of an enterprise that connects technology, people, data, compliance, and risk into a single traversable surface.

The value of approaching the CDAIO practicum this way comes down to three things the graph can do that traditional analysis cannot.

8.1 Instant Situational Reporting

Rather than spending weeks compiling a status deck, you can ask the graph directly. These are real queries you can run today through Cowork or Claude Desktop — no manual assembly required:

"What is the current state of the knowledge graph?".#The `get_statistics` tool returns entity and relationship counts by type, graph density, and connectivity status. One question, one answer, and you have a structural snapshot of the entire enterprise model.

"Which entities are the most connected?".#The `compute_centrality` tool with PageRank or betweenness metrics identifies the nodes that matter most — the systems, people, or departments where failure or compromise would cascade the furthest. This is the kind of insight that takes weeks to surface through interviews and usually arrives too late.

"What would be affected if this system went down?".#The `get_blast_radius` tool traces every entity reachable within N hops of a given node. You get a dependency map in seconds that would otherwise require a cross-functional war room to reconstruct.

8.2 Gap Analysis and Enrichment Recommendations

The graph is designed to surface what is missing, not just what is present. Once you understand the schema and the evidence classification system, identifying gaps becomes conversational:

"Show me all entities of type 'data_asset' and their relationships.".#If a data asset exists but has no `governed_by` relationship to a policy or no `owned_by` relationship to a person, that is a governance gap. The graph makes the absence visible because the structure demands the relationship exist.

"Which entities have a confidence level below 0.7?".#A search filtered by provenance metadata surfaces everything that is still at the HYPOTHESIS tier. These are the enrichment priorities — the claims that need corroboration before they can be treated as reliable.

"What entity types have fewer than five instances?".#Sparse entity types signal underexplored domains. If the graph has 200 systems but only 3 data flows, the data architecture layer is undermodeled relative to the technology layer. That imbalance is a research directive.

8.3 Adding and Enriching Without Manual Effort

None of this requires manual JSON editing, command-line gymnastics, or separate validation runs. Through Cowork, you interact with the graph in natural language and the system handles every downstream mechanic — schema validation, UUID generation, relationship constraint checking, file persistence, git staging, commit, and push.

Adding an entity: "Add a new system entity called 'Salesforce CRM' with status active, classification confidential, and source the 2024 10-K filing." Cowork creates the entity via `add_entity`, validates it against the schema, persists it, and commits the change to your branch. You never touch a JSON file.

Adding a relationship: "Create a 'depends_on' relationship from the ERP system to the Salesforce CRM with weight 0.8 and confidence 0.85." The `add_relationship` tool validates the source and target exist, confirms the relationship type is valid for those entity types via the schema constraints, and writes it. If it would violate domain/range rules, it rejects it and tells you why.

Enriching existing entities: "Update the Oracle EBS entity to include patch status 'current' and add the 2025 SOC 2 report as the primary data source." The `update_entity` tool applies the changes and validates the result.

Batch operations: "Add these five vendor entities and create 'supplies' relationships to the procurement department for each." The `add_relationships_batch` tool validates all inputs before committing any — if one fails validation, nothing is written, so you never end up with a half-complete batch.

Document generation: "Generate a Word document summarizing all system entities, their dependencies, and their compliance status." Cowork queries the graph, structures the output, and produces a `.docx` file ready for submission or presentation — directly from the conversation.

The automation daemons handle the rest. Your changes propagate to the per-type JSON files within 30 minutes, sync to your personal branch, and surface in the evening PR for review. The feedback loop from "I noticed something is missing" to "it is now in the graph and validated" can be measured in minutes, not sprints.

9 Questions for the CMU CDAIO Practicum

The following questions map directly to the five program deliverables and the frameworks covered in class. Each can be answered — or at least substantially advanced — by querying and enriching the knowledge graph. They are organized by the course modules that motivate them.

9.1 Deliverable 1: Company Overview and Current State

These questions establish the baseline. If the graph cannot answer them, the gaps themselves are the first finding.

- “List all organizational units and their reporting relationships. Where are there structural gaps — departments with no parent or no people assigned?”
- “What systems does the company operate? Which have no 'owned_by' relationship to a department or person?” This surfaces shadow IT and governance blind spots.
- “Show me every vendor entity and its contractual relationships. Which vendors have no contract entity linked?” An incomplete vendor-to-contract map is a procurement risk.
- “What is the geographic footprint? List all site and jurisdiction entities and their regulatory relationships.” This is the starting point for understanding where compliance obligations apply.

9.2 Deliverable 2: Capability and Maturity Assessment

These questions align with the DMBOK, DCAM, and TDWI frameworks discussed in Doug Laney's session. The graph encodes the structural evidence that a maturity assessment depends on.

- “For each of the 11 architectural layers, how many entities and relationships exist? Which layers are the most sparse relative to the others?” Layer imbalance is a maturity signal — a company with 200 systems but 3 data flows has not modeled its data architecture.
- “Which data assets have no 'governed_by' relationship to a regulation, policy, or control?” This is a direct measure of data governance maturity. The DCAM framework treats governance traceability as a core capability.

- “What percentage of entities are at the FACT confidence tier versus INFERENCE or HYPOTHESIS?” A graph dominated by hypotheses is an early-stage assessment. One dominated by facts has been through rigorous corroboration. This is a maturity indicator in itself.
- “Show me all controls and their 'mitigates' relationships to risks. Which risks have no mitigating control?” Unmitigated risks are the gap analysis that security and compliance maturity models demand.

9.3 Deliverable 3: Data and AI Strategy

These questions reflect Krishna Cheriath's emphasis on use-case-driven strategy and CFO-certifiable value.

- “Which systems are the most connected by 'depends_on' relationships? Compute centrality by PageRank.” The most central systems are where an AI initiative or data platform investment would have the highest leverage — and the highest blast radius if it fails.
- “What data assets exist and what business capabilities do they support? Are there capabilities with no supporting data asset?” A capability without data is a strategic gap. This is the beginning of a use-case prioritization conversation.
- “Trace the path from a specific business capability to the systems, data assets, and people that support it.” This is the kind of end-to-end traceability that a CDAIO needs to build a credible business case — showing the CEO exactly how data connects to the investor promises.
- “Which data assets are classified as confidential or restricted but have no encryption or access control relationship?” This is where data strategy meets risk quantification in financial terms, not FUD.

9.4 Deliverable 4: Strategic Plan for Enterprise Data Management

These questions support the roadmap and implementation planning that the midterm deliverable requires.

- “What is the blast radius of the top 5 most connected systems? If we decommission or modernize one, what else is affected?” This is infrastructure planning grounded in dependency analysis rather than guesswork.
- “Show me all initiatives and their relationships to systems, departments, and data assets. Which initiatives overlap in scope?” Overlap detection prevents duplicated effort and conflicting transformation programs.

- “Which entity types have the highest ratio of HYPOTHESIS-tier provenance? Rank by enrichment priority.” This tells you where to invest research effort next — the areas where the assessment is thinnest.
- “Generate a summary of all relationships of type 'subject_to' between systems and regulations. Which systems are under the heaviest regulatory burden?” Regulatory load mapping drives prioritization for compliance automation and governance investment.

9.5 Deliverable 5: Final Presentation

These questions help produce the artifacts and narratives that a final presentation requires.

- “Give me a statistical overview of the graph: entity counts by type, relationship counts by type, overall density, and connectivity.” This is the executive summary slide — one query, one answer.
- “What are the top 10 entities by betweenness centrality and what do they connect?” Betweenness centrality finds the bottlenecks and bridges. These are the entities that appear in every critical path and deserve the most attention in a presentation.
- “Compare the current graph coverage to the 11 architectural layers defined in the schema. Which layers have the strongest coverage and which have the weakest?” This is the maturity heatmap that boards and executives understand immediately.
- “Generate a report of all enrichment actions taken this month — entities added, relationships created, confidence levels upgraded — with provenance for each.” This is the evidence trail that demonstrates rigor and velocity. It shows the practicum is not a static document but a living, evolving model.

10 Key Principles to Internalize

Provenance is not metadata — it is the product. The graph is only as trustworthy as the evidence chain behind each claim. Document your sources rigorously.

Confidence is not a formality. Distinguishing FACT from INFERENCE from HYPOTHESIS is how we maintain intellectual integrity at scale. Be honest about what you know versus what you are estimating.

Schema enforcement exists for a reason. The validation gates are not bureaucracy. They prevent the kind of data quality erosion that makes knowledge graphs useless six months after they are built.

Search before you create. Duplicates are the most common contributor error and the hardest to clean up after the fact. The 30 seconds you spend running validation saves hours of deduplication downstream.

The per-type file structure is your friend. It is designed so that parallel contributions do not conflict. Respect the structure and you will rarely have a merge conflict.

11 Questions?

Start by reading the full README in each repository. The `ARCHITECTURE.md` in `hc-enterprise-kg` is particularly worth your time — it documents every major design decision and the rationale behind it. For contribution specifics, the `CONTRIBUTING.md` in `hc-cdaio-kg` covers the complete validation workflow.

Welcome aboard.